



C++ - 모듈 02

임시 다형성, 연산자 오버로딩
정통 정식 수업 형태

요약:

이 문서에는 C++ 모듈의 모듈 02 연습이 포함되어 있습니다.

버전: 8

내용물

나	소개	2
II	일반 규칙	삼
III	새로운 규칙	5
IV	연습 00: 정통 정식 형식의 나의 첫 수업	6
V	연습 01: 보다 유용한 고정 소수점 수 클래스를 향하여	8
VI	연습 02: 이제 우리는 이야기 중입니다.	10
VII	연습 03: BSP	12
VIII	제출 및 동료 평가	14

제1장

소개

C++는 Bjarne Stroustrup이 C 프로그래밍 언어 또는 "C with Classes"의 확장으로 만든 범용 프로그래밍 언어입니다(출처: [위키피디아](#)).

이 모듈의 목표는 다음을 소개하는 것입니다. **객체 지향 프로그래밍**. 이것이 C++ 여정의 출발점이 될 것입니다. OOP를 배우려면 많은 언어를 권장합니다. C++는 오랜 친구인 C에서 파생되었기 때문에 선택하기로 결정했습니다. C++는 복잡한 언어이고 단순함을 유지하기 위해 코드는 C++98 표준을 준수합니다.

우리는 현대 C++가 여러 측면에서 많이 다르다는 것을 알고 있습니다. 따라서 능숙한 C++ 개발자가 되고 싶다면 42 Common Core 이후에 더 나아가는 것은 여러분에게 달려 있습니다!

제2장

일반 규칙

컴파일 중

- 다음을 사용하여 코드를 컴파일하세요. `g++` 그리고 깃발 `-Wextra -Werror`
- 플래그를 추가해도 코드는 계속 컴파일되어야 합니다. 표준 = C++98

형식 및 명명 규칙

- 연습 디렉토리의 이름은 다음과 같이 지정됩니다. `ex00`, `ex01`, ... , 엑스
- 지침에서 요구하는 대로 파일, 클래스, 함수, 멤버 함수 및 속성의 이름을 지정하십시오.
- 클래스 이름을 적습니다. **어퍼케멜케이스** 체제. 클래스 코드가 포함된 파일의 이름은 항상 클래스 이름에 따라 지정됩니다. 예를 들어:
클래스 이름. `hpp`/클래스 이름. `h`, 클래스 이름. `cpp`, 또는 클래스명. `tpp`. 그런 다음 벽돌 벽을 나타내는 "BrickWall" 클래스 정의가 포함된 헤더 파일이 있는 경우 해당 이름은 다음과 같습니다. `BrickWall.hpp`.
- 달리 지정하지 않는 한, 모든 출력 메시지는 개행 문자로 끝나야 하며 표준 출력에 표시되어야 합니다.
- *안녕 노르미네트!* C++ 모듈에는 코딩 스타일이 적용되지 않습니다. 당신이 좋아하는 것을 따라갈 수 있습니다. 하지만 동료 평가자가 이해할 수 없는 코드는 채점할 수 없는 코드라는 점을 명심하세요. 깨끗하고 읽기 쉬운 코드를 작성하기 위해 최선을 다하십시오.

허용/금지

더 이상 C로 코딩하지 않습니다. C++를 배울 시간입니다! 그러므로:

- 표준 라이브러리의 거의 모든 것을 사용할 수 있습니다. 따라서 이미 알고 있는 내용을 고수하는 대신 익숙한 C 함수의 C++ 버전을 최대한 많이 사용하는 것이 현명할 것입니다.
- 그러나 다른 외부 라이브러리는 사용할 수 없습니다. 이는 C++11(및 파생 형식)을 의미하며 후원도서관은 금지되어 있습니다. 다음 기능도 금지됩니다. `*printf()`, `*alloc()` 그리고 무로(`).` 사용하면 성적이 0이 되고 그게 전부입니다.

- 별도로 명시하지 않는 한, 네임스페이스 <ns_name> 사용 그리고 친구 키워드는 금지되어 있습니다. 그렇지 않으면 성적은 -42가 됩니다.
- **모듈 08과 09에서만 STL을 사용할 수 있습니다.** 즉, **아니오 컨테이너**(벡터/목록/맵 등) 및 **아니오 알고리즘**(<를 포함해야 하는 모든 것 알고리즘>헤더) 그때까지. 그렇지 않으면 성적은 -42가 됩니다.

몇 가지 디자인 요구 사항

- C++에서도 메모리 누수가 발생합니다. 메모리를 할당할 때(새로운 키워드)를 피해야 합니다. **메모리 누수.**
- 모듈 02부터 모듈 09까지 수업은 다음과 같이 설계되어야 합니다. **달리 명시적으로 언급된 경우를 제외하고 정교회 정경 형식.**
- 헤더 파일에 있는 모든 함수 구현(함수 템플릿 제외)은 실습에서 0을 의미합니다.
- 각 헤더를 다른 헤더와 독립적으로 사용할 수 있어야 합니다. 따라서 필요한 모든 종속성을 포함해야 합니다. 그러나 다음을 추가하여 이중 포함 문제를 피해야 합니다. **경비원을 포함하다.** 그렇지 않으면 성적은 0이 됩니다.

나를 읽어주세요

- 필요한 경우(예: 코드 분할) 몇 가지 추가 파일을 추가할 수 있습니다. 이러한 과제는 프로그램에 의해 확인되지 않으므로 필수 파일을 제출하는 한 자유롭게 확인하십시오.
- 때로는 연습 지침이 짧아 보이지만 지침에 명시적으로 기록되지 않은 요구 사항이 예제를 통해 표시될 수 있습니다.
- 시작하기 전에 각 모듈을 완전히 읽으십시오! 정말로, 그렇게 하세요.
- 오딘이, 토르가! 머리를 써라!!!



많은 클래스를 구현해야 합니다. 즐겨 사용하는 텍스트 편집기를 스크립트로 작성할 수 없다면 이는 지루해 보일 수 있습니다.



연습을 완료할 수 있는 어느 정도의 자유가 주어집니다. 그러나 필수 규칙을 따르고 게으르지 마십시오. 당신은 많은 유용한 정보를 놓칠 것입니다! 주저하지 말고 이론적 개념에 대해 읽어보세요.

제3장

새로운 규칙

이제부터 모든 수업은 다음과 같이 설계되어야 합니다. **정통 정경 형식**, 달리 명시적으로 언급되지 않는 한. 그런 다음 아래의 네 가지 필수 멤버 함수를 구현합니다.

- 기본 생성자
- 복사 생성자
- 복사 할당 연산자
- 오물 소각로

수업 코드를 두 개의 파일로 분할합니다. 헤더 파일(.hpp/.h)에는 클래스 정의가 포함되어 있는 반면 소스 파일(.cpp)에는 구현이 포함되어 있습니다.

제4장

연습 00: 정통 정식 형식의 나의 첫 수업

	운동 : 00
정통 정식 형식의 나의 첫 수업	
제출 디렉토리: 전-00/	
제출할 파일: Makefile, main.cpp, 고정.{h, hpp}, 고정.cpp	
금지된 기능: 없음	

당신은 정수와 부동 소수점 숫자를 알고 있다고 생각합니다. 얼마나 귀엽습니까?

이 3페이지 분량의 기사를 읽어보세요([1](#),[2](#),[3](#)) 당신이 그렇지 않다는 것을 발견하기 위해. 계속해서 읽어보세요.

오늘까지 코드에 사용된 모든 숫자는 기본적으로 정수, 부동 소수점 숫자 또는 그 변형(짧은, 문자, 긴, 더블, 기타 등등). 위의 기사를 읽은 후에는 정수와 부동 소수점 숫자가 반대 특성을 가지고 있다고 가정하는 것이 안전합니다.

하지만 오늘은 상황이 바뀔 것입니다. 새롭고 멋진 숫자 유형을 발견하게 될 것입니다: **고정 소수점 숫자**! 대부분의 언어의 스칼라 유형에서 영원히 누락된 고정 소수점 숫자는 성능, 정확성, 범위 및 정밀도 간의 귀중한 균형을 제공합니다. 이는 고정 소수점 숫자가 특히 컴퓨터 그래픽, 사운드 처리 또는 과학 프로그래밍에 적용 가능한 이유를 설명합니다.

C++에는 고정 소수점 숫자가 없기 때문에 고정 소수점 숫자를 추가할 것입니다. [이 기사](#) 버클리에서의 출발은 좋은 것 같아요. Berkeley University가 무엇인지 모른다면 다음을 읽어보세요. [이 구역](#) Wikipedia 페이지의 내용입니다.

고정 소수점 숫자를 나타내는 정통 정식 형식의 클래스를 만듭니다.

- 비공개 회원:
 - **안정수** 고정 소수점 숫자 값을 저장합니다.
 - **정적 상수 정수** 소수 비트 수를 저장합니다. 그 값은 항상 정수 리터럴입니다.8.
- 공개 회원:
 - 고정 소수점 숫자 값을 초기화하는 기본 생성자0.
 - 복사 생성자.
 - 복사 할당 연산자 오버로드입니다.
 - 소멸자.
 - 멤버 함수 `int getRawBits( void ) const`; 고정 소수점 값의 원시 값을 반환합니다.
 - 멤버 함수 `void setRawBits( int const raw )`; 고정 소수점 숫자의 원시 값을 설정합니다.

이 코드를 실행하면:

```
# 포함하다 <아이오스트림>

정수 기본(무효의) {

    고정됨;
    고정 b(a);
    고정 c;

    c = b;

    std::cout << a.getRawBits() << std::endl; std::cout
    << b.getRawBits() << std::endl; std::cout <<
    c.getRawBits() << std::endl;

    반환0;
}
```

다음과 유사한 내용이 출력되어야 합니다.

```
$> ./a.out
기본 생성자 호출 복사 생성자 호출

복사 할당 연산자 // <-- 호출된 getRawBits 멤버 함수 구현에 따라 이 줄이 누락될 수 있습니다.

기본 생성자 호출 할당 연산자 호출
getRawBits 멤버 함수 getRawBits 멤버 함
수 호출 0

getRawBits 멤버 함수 호출 0

getRawBits 멤버 함수 호출 0

소멸자가 호출됨
소멸자가 호출됨
소멸자가 호출됨
$>
```

제5장

연습 01: 보다 유용한 고정 소수점 수 클래스를 향하여

	운동 01
보다 유용한 고정 소수점 수 클래스를 향하여	
제출 디렉토리: 전-01/	
제출할 파일: Makefile, main.cpp, 고정.{h, hpp}, 고정.cpp	
허용되는 기능: roundf(<cmath>에서)	

이전 연습은 좋은 시작이었지만 우리 수업은 꽤 쓸모가 없습니다. 값만 나타낼 수 있습니다. 0.0.

다음 공용 생성자 및 공용 멤버 함수를 클래스에 추가합니다.

- 다음을 수행하는 생성자 **상수 정수** 매개변수로. 이를 해당 고정 소수점 값으로 변환합니다. 분수 비트 값은 연습 00과 같이 8로 초기화됩니다.
- 다음을 수행하는 생성자 **상수 부동 소수점 수** 매개변수로. 이를 해당 고정 소수점 값으로 변환합니다. 분수 비트 값은 연습 00과 같이 8로 초기화됩니다.
- 멤버 함수 `float toFloat( void ) const`; 고정 소수점 값을 부동 소수점 값으로 변환하는 것입니다.
- 멤버 함수 `int toInt( void ) const`; 고정 소수점 값을 정수 값으로 변환합니다.

그리고 다음 함수를 **결정된** 클래스 파일:

- 매개변수로 전달된 출력 스트림 개체에 고정 소수점 숫자의 부동 소수점 표현을 삽입하는 삽입(`<<`) 연산자의 오버로드입니다.

이 코드를 실행하면:

```
# 포함하다<아이오스트림>

정수 기본(무효의) {

    결정된      t;
    결정된const비(10); 결정된
    const씨(42.42f); 결정된const
    d(b);

    a = 고정(1234.4321f);

    표준::cout <<"a는 " << a << std::endl; 표준::cout
    <<"b는 " << b << std::endl; 표준::cout <<"c는 " << c
    << std::endl; 표준::cout <<"d는 " << d <<
    std::endl;

    표준::cout <<"a는 " << a.toInt() <<"정수로" << 표준::endl; 표준::cout <<"b는 " <<
    b.toInt() <<"정수로" << 표준::endl; 표준::cout <<"c는 " << c.toInt() <<"정수로" << 표
    준::endl; 표준::cout <<"d는 " << d.toInt() <<"정수로" << 표준::endl;

    반쯤0;

}
```

다음과 유사한 내용이 출력되어야 합니다.

```
$> ./a.out
Int 생성자라는 기본 생성자가 호출
되었습니다.
Float 생성자 Copy 생성자 호출 Copy 할당
연산자 호출 Float 생성자

소멸자라고 불리는 복사 할당 연산자가 호출
됩니다.
a는 1234.43이다
b는 10이다
c는 42.4219입니다.
d는 10이다
a는 정수로서 1234입니다.
b는 정수로서 10입니다. c는
정수로서 42입니다. d는 정
수로서 10입니다. 소멸자는
호출됩니다.
소멸자가 호출됨
소멸자가 호출됨
소멸자가 호출됨
$>
```

제6장

연습 02: 이제 우리는 이야기 중입니다.

	운동 02
지금 우리는 이야기하고 있어요	
제출 디렉토리: 전-02/	
제출할 파일: Makefile, main.cpp, 고정.{h, hpp}, 고정.cpp	
허용되는 기능: roundf(<cmath>에서)	

다음 연산자를 오버로드하려면 클래스에 공용 멤버 함수를 추가하세요.

- 6가지 비교 연산자: $>$, $<$, $>=$, $<=$, $==$ 및 $!=$.
- 4개의 산술 연산자: $+$, $-$, $*$, $/$.
- 4개의 증가/감소(사전 증가 및 사후 증가, 사전 감소 및 사후 감소) 연산자는 표현 가능한 가장 작은 고정 소수점 값을 늘리거나 줄입니다. ϵ 예를 들어 $1 + \epsilon > 1$.

다음 네 가지 공용 오버로드된 멤버 함수를 클래스에 추가하세요.

- 정적 멤버 함수 분 고정 소수점 숫자에 대한 두 개의 참조를 매개변수로 사용하고 가장 작은 참조에 대한 참조를 반환합니다.
- 정적 멤버 함수 분 매개변수로 두 개의 참조를 취합니다. **끊임없는** 고정 소수점 숫자를 반환하고 가장 작은 숫자에 대한 참조를 반환합니다.
- 정적 멤버 함수 최대 고정 소수점 숫자에 대한 두 개의 참조를 매개변수로 사용하고 가장 큰 참조에 대한 참조를 반환합니다.
- 정적 멤버 함수 최대 매개변수로 두 개의 참조를 취합니다. **끊임없는** 고정 소수점 숫자를 선택하고 가장 큰 숫자에 대한 참조를 반환합니다.

수업의 모든 기능을 테스트하는 것은 여러분에게 달려 있습니다. 그러나 아래 코드를 실행하십시오.

```
# 포함하다 <아이오스트림>

정수 기본(무효의){

    결정된      t;
    결정된const  b( 고정(5시 05분) * 고정(2) );

    std::cout << a << std::endl; std::cout
    << ++a << std::endl; std::cout << a <<
    std::endl; std::cout << a++ <<
    std::endl; std::cout << a << std::endl;

    std::cout << b << std::endl;

    std::cout << 고정::max( a, b ) << std::endl;

    반환0;
}
```

다음과 같이 출력해야 합니다(가독성을 높이기 위해 아래 예에서는 생성자/소멸자 메시지가 제거되었습니다).

```
$> ./a.out
0
0.00390625
0.00390625
0.00390625
0.0078125
10.1016
10.1016
$>
```



0으로 나누는 경우 프로그램이 충돌하는 것은 허용됩니다.

제7장

연습 03: BSP

	운동 03
	BSP
제출 디렉토리: 전-03/	
제출할 파일: Makefile, main.cpp, 고정.{h, hpp}, 고정.cpp, Point.{h, hpp}, Point.cpp, bsp.cpp	
허용되는 기능: roundf(<cmath>에서)	

이제 기능이 생겼으니 **결정된** 수업, 활용하시면 좋을 것 같습니다.

점이 삼각형 내부에 있는지 여부를 나타내는 함수를 구현합니다. 매우 유용하지 않나요?



BSP는 바이너리 공간 파티셔닝을 나타냅니다. 물론이죠. :)



연습 03을 수행하지 않고도 이 모듈을 통과할 수 있습니다.

클래스를 만드는 것부터 시작해 보겠습니다. **가리키다** 2D 포인트를 나타내는 정통 정식 형식:

- 비공개 회원:
 - 고정 const 속성엑스.
 - 고정 const 속성와이.
 - 다른 유용한 것.
- 공개 회원:
 - 초기화하는 기본 생성자엑스그리고와이에게0.
 - 두 개의 상수 부동 소수점 숫자를 매개변수로 사용하는 생성자입니다. 초기화합니다엑스그리고와이해당 매개변수를 사용하여
 - 복사 생성자.
 - 복사 할당 연산자 오버로드입니다.
 - 소멸자.
 - 다른 유용한 것.

결론적으로 적절한 파일에 다음 함수를 구현하십시오.

```
bool bsp(포인트 const a, 포인트 const b, 포인트 const c, 포인트 const 포인트);
```

- a, b, c: 우리가 사랑하는 삼각형의 꼭지점입니다.
- point : 확인할 점.
- 반환값: 점이 삼각형 내부에 있으면 참입니다. 그렇지 않으면 거짓입니다.
따라서 점이 정점이거나 가장자리에 있으면 False를 반환합니다.

클래스가 예상대로 작동하는지 확인하기 위해 자체 테스트를 구현하고 설정하십시오.

제8장

제출 및 동료 평가

과제를 제출하세요. 힘내평소대로 저장소. 방어 중에는 저장소 내부 작업만 평가됩니다. 주저하지 말고 폴더와 파일의 이름이 올바른지 다시 확인하세요.



????????????? XXXXXXXXXXXX = \$3\$d6f957a965f8361750a3ba6c97554e9f